

Wireless Security Lab 1 - GNSS Spoofing

Claudio Foroncelli [ID: 360300]

Marco Salza [ID: 362291]

Marco Canavese [ID: 362258]

Sattar Aftabi Amaneh [ID: 336778]

Politecnico di Torino

Abstract is missing, more than 3 pages.

1 INTRODUCTION

This paper presents an experimental analysis of Global Navigation Satellite System (GNSS) data, with a focus on signal behavior and spoofing vulnerabilities. Geo-localization data were collected using a smartphone and analyzed under different environmental conditions and smartphone settings. In addition, software-based spoofing techniques were applied to evaluate their impact on positioning accuracy and system reliability.

2 METHODOLOGY

2.1 Devices and tools used

GNSS data were collected using a Google Pixel 9 using a custom OS called GrapheneOS [4]. Raw GNSS measurements were acquired through the GNSS Logger application (version 3.1.1.2), downloaded from the Google Play Store [3].

2.2 Data collection procedure

Measurements were acquired in open-sky environments around the Politecnico di Torino to ensure good satellite visibility. Two types of datasets were recorded: static (device kept still) and dynamic (user running). Static acquisitions were performed under three conditions: nominal settings, with battery-saving mode enabled, and during an active phone call.

The raw data were stored as GNSS log files and processed offline using MATLAB scripts using a modified version of Google's open-source GPS Measurement Tools [2], allowing the simulation of spoof attacks, following the standard processing workflow described in the lab material.

2.3 Spoofing setup

Different spoofing configurations were tested in order to evaluate how parameter variations affect the resulting position estimates and signal observables.

The spoofed position was selected as a point geographically close to the true receiver trajectory, allowing for realistic motion while simultaneously generating measurable effects on the PVT solution.

A constant spoof delay of 0.5 s was applied in all experiments to emulate a replay-based spoofing (meaconing) attack, in which authentic GNSS signals are retransmitted with a fixed latency. The specific value of 0.5 s was selected as an experimental parameter: it is sufficiently large to generate clearly observable effects in the pseudorange and clock-bias estimates, while ensuring consistency across all tested spoofing configurations.

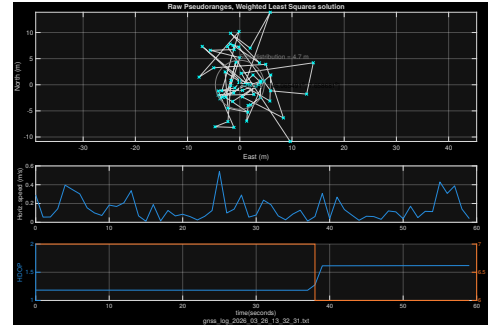


Figure 1: Nominal conditions - Raw pseudoranges

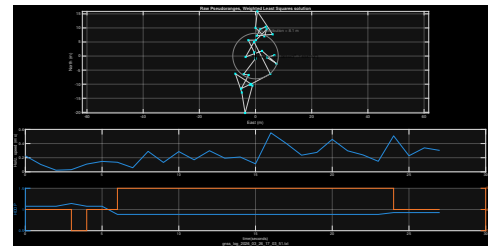


Figure 2: Battery saver mode - Raw pseudoranges

3 RESULTS AND DISCUSSION

3.1 Impact of Device Operating Conditions

3.1.1 Effect of Battery Saver Mode. To evaluate the impact of device operating conditions on GNSS performance, two datasets were collected along the same trajectory: one under nominal conditions and one with battery saver mode enabled.

Under nominal conditions (Fig. 1), the positioning solution appears stable, with a 50% distribution radius of approximately 4.7 m. The estimated points are tightly clustered around the median position, indicating good precision.

When battery saver mode is enabled (Fig. 2), the positioning performance degrades, with the median error increasing to approximately 8.1 m. The solution becomes more dispersed, reflecting reduced estimation precision.

The observed degradation is associated with a reduced number of collected samples, as battery saver mode lowers the GNSS measurement acquisition rate and therefore provides fewer observations to the positioning algorithm. Both recordings last 60 seconds: the nominal dataset contains 59 samples (approximately 1 sample/s), while the battery saver dataset contains 28 samples (approximately 0.5 sample/s), corresponding to an almost halved sampling rate.

This reduction may make the estimation less robust and more sensitive to measurement noise.

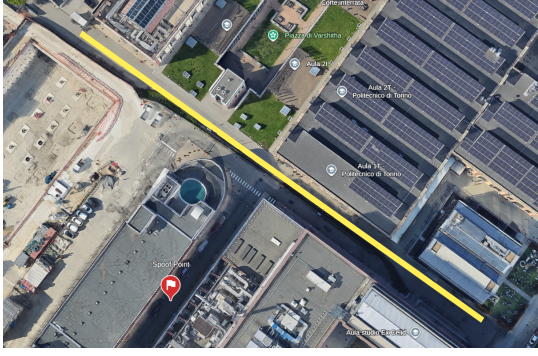


Figure 3: True trajectory and selected spoofing target position

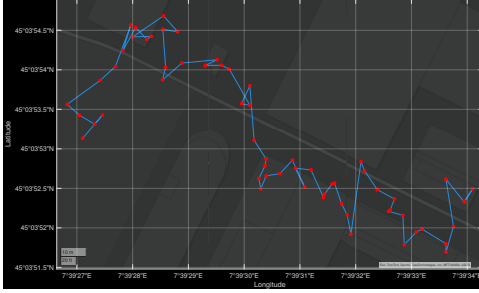


Figure 4: Path reconstructed on MATLAB

Conclusion: battery saver mode may degrade GNSS positioning precision. The reduced sampling rate is a plausible contributing factor, since fewer measurements are available for the PVT estimation.

3.2 Spoofing Analysis

3.2.1 Spoofing Implementation. To evaluate spoofing under realistic conditions, a dynamic dataset was collected along a predefined trajectory in an open-sky environment of approximately 175 m, shown in Fig. 3. The reconstruction of the sampled path, recorded by the smartphone, can be seen in Fig. 4.

A spoofing target position was selected close to the true trajectory (shown in Fig. 3 as a red placemark), allowing a measurable deviation while maintaining a realistic scenario. This setup enables a direct comparison between the true and spoofed solutions.

3.2.2 Reducing Spoofing Detectability via Software Modification. To reduce detectability, the spoofing algorithm was extended with a stop mechanism that limits the attack to a predefined time. This allows the receiver to return smoothly to nominal operation after the spoofing phase.

In `ProcessGnssMeasScript.m`, the spoofing stop time is defined as:

```
1 spoof.t_stop = 40;
```

The corresponding indices are computed as:

```
1 idxStop = find(timeSeconds <= spoof.t_stop, 1, 'last');
```

The spoofing is then applied only within this interval:

```
1 if i >= idxStart && i <= idxStop
```

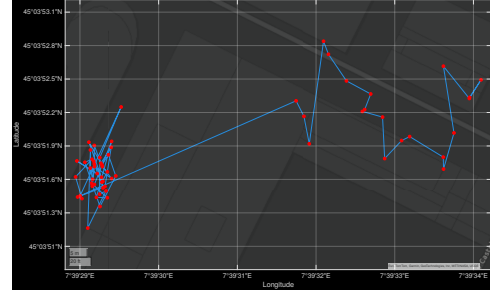


Figure 5: Spoofing without stop time

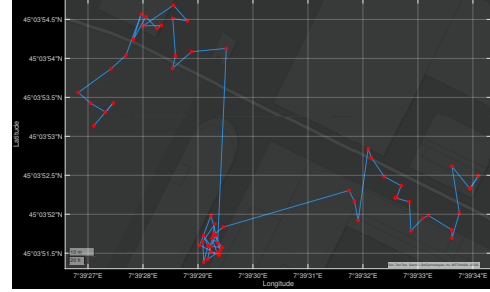


Figure 6: Spoofing with stop time

In the original implementation, spoofing was applied abruptly, resulting in an unrealistic jump in the estimated position, as shown in Fig. 5. To improve realism and reduce detectability, a gradual transition mechanism was introduced.

3.2.3 Interpolation-Based Spoofing. The transition from the true receiver position to the spoofed position is modeled using a time-varying interpolation parameter $\alpha(t)$ such that:

$$0 \leq \alpha(t) \leq 1$$

where $\alpha = 0$ corresponds to the true position and $\alpha = 1$ to the spoofed target.

The receiver position evolves as:

$$\mathbf{x}_{\text{spooft}}(t) = \mathbf{x}_{\text{true}} + \alpha(t)(\mathbf{x}_{\text{target}} - \mathbf{x}_{\text{true}})$$

Since GNSS positioning is based on pseudoranges, the interpolation is applied at the measurement level:

$$\rho_i^{\text{spooft}}(t) = \rho_i^{\text{true}}(t) + \alpha(t) \Delta \rho_i$$

where $\Delta \rho_i$ represents the pseudorange difference between spoofed and true positions.

By gradually increasing $\alpha(t)$, the receiver is smoothly driven toward the spoofed location, avoiding abrupt discontinuities. When the stop mechanism is enabled, $\alpha(t)$ is symmetrically reduced, ensuring a smooth return to the true trajectory.

This is demonstrated in Fig. 7, where the spoofed position is reached in multiple steps, instead of one straight line.

In practice, the interpolation is implemented as:

```
1 tRxSeconds(J(j)) = tRxSeconds(J(j)) + ...
2   alpha * Pr_diff_spUser(k) + spoof.delay + ...
3   PrSigmaM(J(j))*randn/physconst('lightspeed');
```

Conclusion: interpolation-based spoofing enables smooth and continuous manipulation of the estimated position, significantly

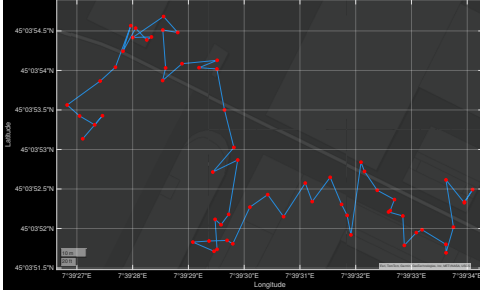


Figure 7: Interpolation-based spoofing

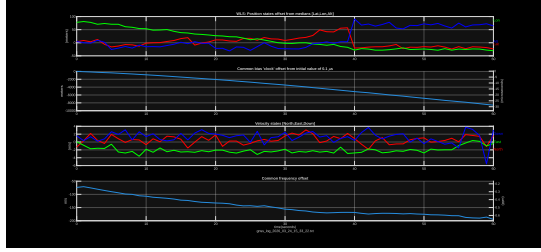


Figure 8: Nominal case

improving realism and reducing detectability compared to abrupt position jumps.

Note: the code was extended using generative AI tools, such as ChatGPT [5], and is available at this GitHub repository [1].

3.2.4 Effect of a Common Spoofing Delay. In a realistic spoofing scenario, the attacker receives authentic GNSS signals and retransmits them toward the victim. This introduces a propagation delay Δt , corresponding to the time required for the signal to travel from the satellites to the spoofer and then to the receiver.

This delay is applied equally to all satellites, so the pseudoranges become:

$$\rho_i^{\text{spooft}} = \rho_i + c \Delta t$$

Since the same offset affects every satellite, the relative geometry is preserved:

$$\rho_i^{\text{spooft}} - \rho_j^{\text{spooft}} = \rho_i - \rho_j$$

Thus, the spatial configuration of the satellites with respect to the receiver remains unchanged, and the estimated position is unaffected.

Velocity depends on the time variation of pseudoranges:

$$v = \frac{d\rho_i}{dt}$$

Because Δt is constant:

$$\frac{d}{dt}(c \Delta t) = 0$$

the pseudorange rates are unchanged, and therefore the estimated velocity is also unaffected.

$$v^{\text{spooft}} = \frac{d\rho_i^{\text{spooft}}}{dt} = \frac{d\rho_i}{dt} + \frac{d}{dt}(c \Delta t) = \frac{d\rho_i}{dt} + 0 = \frac{d\rho_i}{dt} = v$$

The only parameter that absorbs this common offset is the receiver clock bias:

$$\rho_i = \|\mathbf{x} - \mathbf{x}_i\| + c b \Rightarrow b' = b + \Delta t$$

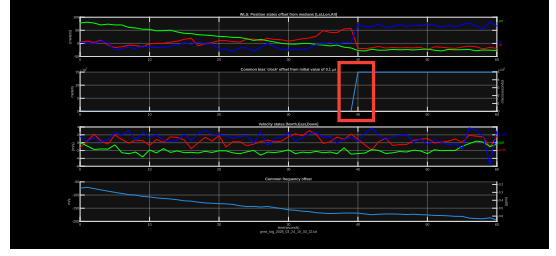


Figure 9: With common delay

The results in Fig. 8 and in Fig. 9 confirm the theoretical analysis. The estimated position and velocity (respectively, first graph and third graph) remain identical in both cases, demonstrating that a common delay does not affect the geometric solution or the pseudorange rates. However, a clear difference appears in the receiver clock drift, which absorbs the imposed delay. In particular, a peak is observed around 40 s (highlighted in red in Fig. 9), corresponding to the moment when the spoofing is activated. This behavior confirms that a uniform timing bias is entirely reflected in the receiver clock parameters.

3.2.5 Effect of Phone Call Activity. To evaluate the impact of performing a phone call during GNSS acquisition, two datasets were compared: one under nominal conditions and one during an active call.

Phone were kept still on the ground or held in hand?

The results show a slight degradation in positioning performance. In the nominal recording, the estimated positions are tightly clustered, with a 50% distribution radius of approximately 4.7 m. In contrast, during the phone call, the solution becomes more dispersed, with a larger 50% distribution radius of about 5.7 m.

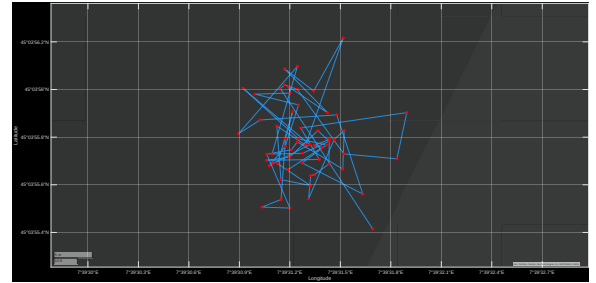


Figure 10: Position scatter under nominal conditions.

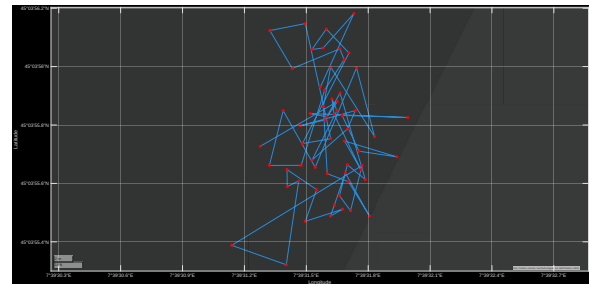


Figure 11: Position scatter during a phone call.

As shown in Fig. 10 and in Fig. 11, the degradation is also evident from the spatial distribution of the estimated positions, which are more spread out during the phone call.

An important aspect to consider is the number of tracked satellites. The nominal recording remains relatively stable, with 6 to 7 satellites, whereas the phone call recording oscillates between 8 and 9 satellites. This is shown in the bottom graphs (represented by the orange line) in Fig. 12 and Fig. 13. Despite the higher number of satellites, this higher variability appears to introduce instability in the positioning solution.

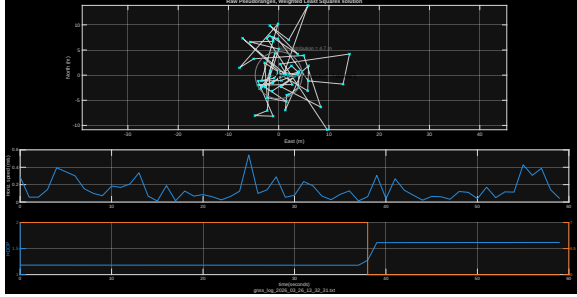


Figure 12: GNSS positioning under nominal conditions.

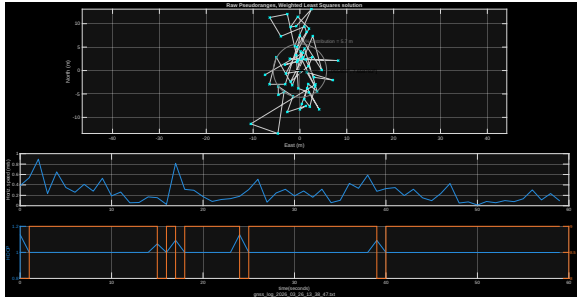


Figure 13: GNSS positioning during a phone call.

Conclusion: the observed degradation in positioning precision during phone call activity cannot be attributed to a single cause. Both potential interference effects and the variability in the number of tracked satellites are likely contributing factors, and further controlled experiments would be required to isolate their individual impact.

4 CONCLUSION

This work presented an experimental analysis of GNSS measurements under different operating conditions, together with the implementation and evaluation of software-based spoofing techniques.

The results show that GNSS positioning performance is strongly influenced by device settings and usage conditions. In particular, battery-saver mode leads to a degradation in positioning precision, mainly due to a reduced sampling rate and variability in satellite tracking. Similarly, performing a phone call during acquisition results in a less stable positioning solution, with increased dispersion of estimated points. However, in both cases, it is not possible to fully isolate the dominant cause of the degradation, as multiple factors (measurement availability, satellite geometry, and potential interference) interact simultaneously.

Moreover, the introduction of a gradual spoofing mechanism based on the α parameter allows the generation of realistic and continuous deviations in the estimated position, significantly reducing detectability compared to abrupt position jumps. However,

while this interpolation-based approach produces smooth position and velocity estimates, it introduces inconsistencies in the timing domain, resulting in a noticeable spike in the receiver clock bias drift.

Additionally, the analysis of a common spoofing delay confirms that such an attack does not affect the estimated position or velocity, but is entirely absorbed by the receiver clock bias, as validated both theoretically and experimentally.

Overall, the results demonstrate that GNSS-based positioning systems are sensitive to both environmental conditions and intentional manipulation, and that even relatively simple spoofing strategies can produce realistic and difficult-to-detect effects.

4.1 Future Work

Further investigations could focus on isolating the individual contributions of different factors affecting positioning accuracy, such as sampling rate, satellite visibility, and device-induced interference. Controlled experiments with fixed satellite configurations would enable a more precise evaluation of these effects.

In addition, an interesting direction is to improve the realism of the spoofing model by moving from interpolation-based manipulation of measurements to a fully trajectory-based approach. While interpolation ensures a smooth position transition, it introduces inconsistencies in the underlying observables: the implied velocity does not match the receiver motion, and the clock bias often exhibits unrealistic peaks during the spoofing phase. These artifacts can be exploited for spoofing detection.

A more advanced approach would consist of generating synthetic measurements from a predefined receiver trajectory, recomputing pseudoranges and Doppler based on satellite geometry and relative motion. This would ensure consistency between position, velocity, and timing, producing signals that evolve coherently over time and making detection significantly more challenging. However, such an approach is considerably more complex to implement, as it requires accurate modeling of satellite positions, receiver dynamics, and measurement generation.

Finally, the development and evaluation of spoofing detection and mitigation techniques represent a natural continuation of this work, contributing to improving the robustness and security of GNSS-based systems.

REFERENCES

- [1] Claudio Foroncelli. 2026. GPS Measurement Tools. (2026). <https://github.com/claudioforoncelli/gps-measurement-tools> GitHub repository.
- [2] Google. 2026. GPS Measurement Tools. (2026). <https://github.com/google/gps-measurement-tools> NSS Measurement Tools code for MATLAB.
- [3] Google LLC. 2026. GnsLogger (Version 3.1.1.2). (2026). <https://play.google.com/store/apps/details?id=com.google.android.apps.location.gps.gnslogger> Android application for logging GNSS measurements.
- [4] GrapheneOS Project. 2026. GrapheneOS. (2026). <https://grapheneos.org/> The private and secure mobile operating system.
- [5] OpenAI. 2026. ChatGPT (GPT-5.3 Instant) conversation. (2026). <https://chatgpt.com/> AI-generated response used for code assistance.